

NEURAL NETWORKS AND DEEP LEARNING

Assignment (2)

Tiny Shakespeare Memory Duel

در پیش‌بینی کاراکتر بعدی Jordan و Elman مقایسه شبکه‌های

دانشگاه شیراز

درس شبکه‌های عصبی و یادگیری عمیق

تمرین دوم

Tiny Shakespeare Memory Duel

برای پیش‌بینی کاراکتر بعدی Jordan و Elman مقایسه مدل‌های

نام و نام خانوادگی: ساغر خردمند حقیقی

شماره دانشجویی: 40464255

استاد: دکتر منصوری

دانشگاه: دانشگاه شیراز

نیمسال تحصیلی: ۱۴۰۳-۱۴۰۴

خلاصه گزارش

پیاده‌سازی character-level به صورت Tiny Shakespeare روی دیتاست Elman و Jordan در این گزارش دو معماری بازگشتی عملکرد بهتری perplexity و loss، accuracy در تمام معیارهای اصلی شامل Elman و مقایسه شده‌اند. نتایج نشان می‌دهد مدل Jordan نسبت به دارد

در این گزارش، دو معماری بازگشتی Elman و Jordan برای مسئله پیش‌بینی کاراکتر بعدی روی دیتاست Tiny Shakespeare پیاده‌سازی و مقایسه شده‌اند. داده‌ها به صورت character-level پردازش شدند و برای کاهش زمان آموزش، از ۳۰٪ دیتاست استفاده شد.

هدف اصلی این پروژه، بررسی تأثیر نوع feedback در شبکه‌های بازگشتی بر عملکرد مدل بود. در مدل Elman، بازخورد از hidden state قبلی گرفته می‌شود، در حالی که در مدل Jordan از خروجی قبلی مدل استفاده می‌شود.

نتایج آزمایش نشان داد که مدل Elman در معیارهای اصلی شامل accuracy، perplexity loss عملکرد بهتری نسبت به مدل Jordan دارد. همچنین بررسی نمونه‌های تولید متن در دماهای مختلف نشان داد که Elman معمولاً خروجی منسجم‌تر و طبیعی‌تری تولید می‌کند. بنابراین، در این آزمایش، معماری Elman برای مدل‌سازی متنی و پیش‌بینی کاراکتر بعدی مناسب‌تر از Jordan بوده است.

p2_completed.py: کد مورد استفاده

٪ درصد داده: ۳۰ - tiny-shakespeare(1).txt داده مورد استفاده

فهرست مطالب

۱. مقدمه
 ۲. هدف تمرین
 ۳. دیتاست و پیش‌پردازش
 ۴. معماری مدل‌ها
 ۵. تنظیمات آزمایش
 ۶. معیارهای ارزیابی
 ۷. نتایج نهایی عددی
 ۸. Elman تحلیل مدل
 ۹. Jordan تحلیل مدل
 ۱۰. تحلیل نمودارها
 ۱۱. مقایسه همگرایی و پایداری
 ۱۲. Temperature تولید متن و اثر
 ۱۳. جمع‌بندی تحلیلی
 ۱۴. نتیجه‌گیری نهایی
- پیوست: خروجی اجرای کد

۱. مقدمه

در این تمرین، هدف اصلی بررسی و مقایسه عملکرد دو معماری کلاسیک شبکه‌های بازگشتی، یعنی **Elman Network** و **Jordan Network**، در مسئله پیش‌بینی کاراکتر بعدی است. این مسئله یکی از کاربردهای مهم مدل‌سازی زبانی محسوب می‌شود؛ زیرا مدل باید با مشاهده چند کاراکتر قبلی، کاراکتر بعدی را پیش‌بینی کند.

متن یک نوع داده دنباله‌ای است و ترتیب کاراکترها در آن اهمیت زیادی دارد. برای مثال، در متن‌های نمایشی مانند آثار شکسپیر، نام شخصیت‌ها، علامت دو نقطه، خط جدید، دیالوگ‌ها و علائم نگارشی دارای الگوهای مشخصی هستند. بنابراین مدل باید بتواند این الگوهای ترتیبی را یاد بگیرد و از اطلاعات قبلی برای پیش‌بینی مرحله بعد استفاده کند.

در این پروژه از دیتاست **Tiny Shakespeare** استفاده شد. این دیتاست شامل متن‌هایی به سبک نمایشنامه‌های شکسپیر است و به همین دلیل برای بررسی مدل‌های دنباله‌ای و تولید متن مناسب محسوب می‌شود. در این تمرین، داده به صورت **character-level** پردازش شد؛ یعنی هر حرف، فاصله، علامت نگارشی و خط جدید به عنوان یک واحد جداگانه در نظر گرفته شد.

تفاوت اصلی دو مدل **Elman** و **Jordan** در نوع بازخورد یا **feedback** آن‌هاست. در مدل **Elman**، اطلاعات گذشته از طریق **hidden state** قبلی وارد مرحله بعد می‌شود. اما در مدل **Jordan**، بازخورد از خروجی قبلی مدل گرفته می‌شود. هدف این تمرین این است که بررسی شود کدام نوع حافظه و بازخورد برای یادگیری الگوهای متنی دیتاست **Tiny Shakespeare** مناسب‌تر است.

۲. هدف تمرین

هدف این تمرین فقط پیاده‌سازی دو مدل بازگشتی نیست، بلکه مقایسه دقیق آن‌ها از جنبه‌های مختلف است. برای این منظور، عملکرد هر دو مدل از نظر معیارهای کمی و کیفی بررسی شد.

- در آموزش و اعتبارسنجی loss روند کاهش
- Character Accuracy دقت پیش‌بینی کاراکتر یا
- PPL یا Perplexity مقدار
- سرعت همگرایی و پایداری آموزش
- یکسان seed کیفیت متن تولیدشده با
- های مختلف روی خروجی متن Temperature اثر

برای اینکه مقایسه منصفانه باشد، هر دو مدل با تنظیمات یکسان آموزش داده شدند؛ بنابراین تفاوت نتایج بیشتر به نوع معماری و نوع **Feedback** مربوط است.

۳. دیتاست و پیش‌پردازش

استفاده شد. با توجه به حجم نسبتاً زیاد داده و برای کاهش زمان آموزش، فقط ۳۰٪ **Tiny Shakespeare** در این پروژه از دیتاست ابتدایی دیتاست مورد استفاده قرار گرفت. این انتخاب مطابق صورت تمرین است، زیرا استفاده از بخشی از داده مجاز بوده و فقط باید درصد استفاده‌شده گزارش شود.

مقدار	مورد
30%	درصد داده استفاده‌شده

تعداد کاراکترهای استفاده شده	334,618
Vocabulary size	63
Train characters	301,156
Validation characters	33,462

ابتدا فایل متنی خوانده شد و سپس ۳۰٪ ابتدایی آن انتخاب شد. بعد از آن، همه کاراکترهای یکتای متن استخراج شدند و برای هر برای تبدیل itos برای تبدیل کاراکتر به عدد و stoi: کاراکتر یک شناسه عددی ساخته شد. دو نداشت اصلی در این مرحله استفاده شد. عدد به کاراکتر

مدل، batch، تقسیم شد. در هر validation و ۱۰٪ برای train سپس متن به دنباله‌ای از اعداد تبدیل شد و با نسبت ۹۰٪ برای دنباله‌هایی با طول ۱۰۰ کاراکتر دریافت می‌کند و هدف آن پیش‌بینی کاراکتر بعدی در هر موقعیت است

۴. معماری مدل‌ها

۴.۱ مدل Elman

قبل به گام بعدی hidden state، یکی از ساده‌ترین و شناخته‌شده‌ترین انواع شبکه‌های بازگشتی است. در این مدل Elman مدل منتقل می‌شود و نقش حافظه داخلی مدل را دارد. بنابراین مدل می‌تواند اطلاعاتی از مراحل قبلی را نگه دارد و از آن برای پیش‌بینی کاراکتر بعدی استفاده کند

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b)$$

$$y_t = W_{hy} h_t$$

در یادگیری Elman منتقل می‌شود. این ویژگی معمولاً باعث می‌شود hidden state در این مدل، حافظه به صورت درونی و از طریق الگوهای ترتیبی ساده و متوسط عملکرد پایداری داشته باشد

۴.۲ مدل Jordan

مدل Jordan از نظر ساختار کلی شبیه مدل Elman است، اما تفاوت اصلی آن در نوع feedback یا بازخورد است. در این مدل، به جای استفاده از hidden state قبلی، خروجی قبلی مدل به مرحله بعد وارد می‌شود.

به عبارت دیگر، مدل Jordan تلاش می‌کند با استفاده از نتیجه پیش‌بینی مرحله قبل، وضعیت مرحله بعد را محاسبه کند. بنابراین خروجی قبلی مدل نقش حافظه را دارد و می‌تواند روی پیش‌بینی کاراکتر بعدی تأثیر بگذارد.

$$h_t = \tanh(W_{xh} x_t + W_{yh} y_{t-1} + b)$$

$$y_t = W_{hy} h_t$$

در اینجا خروجی مرحله قبل می‌تواند بر پیش‌بینی مرحله بعد تأثیر بگذارد. این ویژگی از یک طرف می‌تواند مفید باشد، اما اگر مدل در یک مرحله خطا کند، این خطا ممکن است به مراحل بعدی نیز منتقل شود

۵. تنظیمات آزمایش

در جدول p2_completed.py برای مقایسه عادلانه، هر دو مدل با تنظیمات یکسان آموزش داده شدند. تنظیمات نهایی اجرای کد زیر آمده است

پارامتر	مقدار
Sequence length	100
Batch size	64

Embedding size	64
Hidden size	128
Epochs	15
Steps per epoch	100
Optimizer	Adam
Learning rate	0.001
Gradient clip	1.0
Device	CPU
Seed text	ROMEO:\n
Temperatures	0.4, 0.8, 1.2

معیارهای ارزیابی ۶.

۶.۱ Loss

برای محاسبه خطا از **Cross Entropy Loss** استفاده شد، زیرا مسئله پیش‌بینی کاراکتر بعدی یک مسئله **طبقه‌بندی چندکلاسه** است.

در هر گام زمانی، مدل باید از بین ۶۳ کاراکتر موجود در **vocabulary**، محتمل‌ترین کاراکتر بعدی را پیش‌بینی کند. بنابراین خروجی مدل به‌صورت یک توزیع احتمالی روی تمام کاراکترهای موجود تولید می‌شود و سپس با کاراکتر واقعی مقایسه می‌گردد.

به همین دلیل، استفاده از **Cross Entropy Loss** برای این مسئله مناسب است؛ زیرا این تابع خطا اختلاف بین توزیع پیش‌بینی‌شده توسط مدل و برچسب واقعی را اندازه‌گیری می‌کند.

۶.۲ Character Accuracy

بیشتر باشد، مدل در انتخاب **accuracy** این معیار نشان می‌دهد چند درصد از کاراکترها درست پیش‌بینی شده‌اند. هرچه مقدار **accuracy** کاراکتر درست موفق‌تر بوده است.

$$\text{Accuracy} = \text{Correct Predictions} / \text{Total Predictions}$$

۶.۳ Perplexity

نشان می‌دهد مدل در **PPL** محاسبه می‌شود و یکی از معیارهای رایج در مدل‌سازی زبانی است. مقدار کمتر **loss** از روی **Perplexity** پیش‌بینی کاراکتر بعدی کمتر سردرگم است.

$$\text{PPL} = \exp(\text{loss})$$

نتایج نهایی عددی ۷.

:پانزدهم نشان می‌دهد epoch جدول زیر نتایج نهایی دو مدل را در پایان

Model	Train Loss	Train Acc	Train PPL	Val Loss	Val Acc	Val PPL
Elman	1.5667	53.51%	4.79	1.8389	47.65%	6.29
Jordan	2.0541	39.13%	7.80	2.2143	36.10%	9.16

در طول آموزش به شکل زیر به دست آمد validation همچنین بهترین دقت

Model	Best Val Acc	Epoch
Elman	47.77%	14
Jordan	36.23%	14

نتیجه عددی اصلی

feedback state-hidden در مدل Elman مؤثرتر از feedback output-distribution در مدل Jordan بوده است

Elman تحلیل مدل ۸.

epoch این مدل از 32.13٪ در Validation Accuracy. های ابتدایی روند یادگیری خوبی نشان داد epoch از همان Elman مدل بود epoch 14 آن نیز 47.77٪ در accuracy اول شروع شد و در پایان آموزش به 47.65٪ رسید. بهترین

نیز از 11.58 به 6.29 رسید. این Validation Perplexity از 2.4496 به 1.8389 کاهش یافت و Elman مدل Validation Loss کاهش نشان می‌دهد مدل به مرور توزیع احتمالی بهتری برای کاراکتر بعدی یاد گرفته است

از 26.57٪ به 53.51٪ رسید که نشان‌دهنده یادگیری قابل توجه روی داده آموزشی Elman مدل Train Accuracy، از طرف دیگر نیز validation نیز بسیار شدید نیست، بنابراین مدل تا حد قابل قبولی توانسته روی داده validation و train است. فاصله بین تعمیم پیدا کند

Jordan تحلیل مدل ۹.

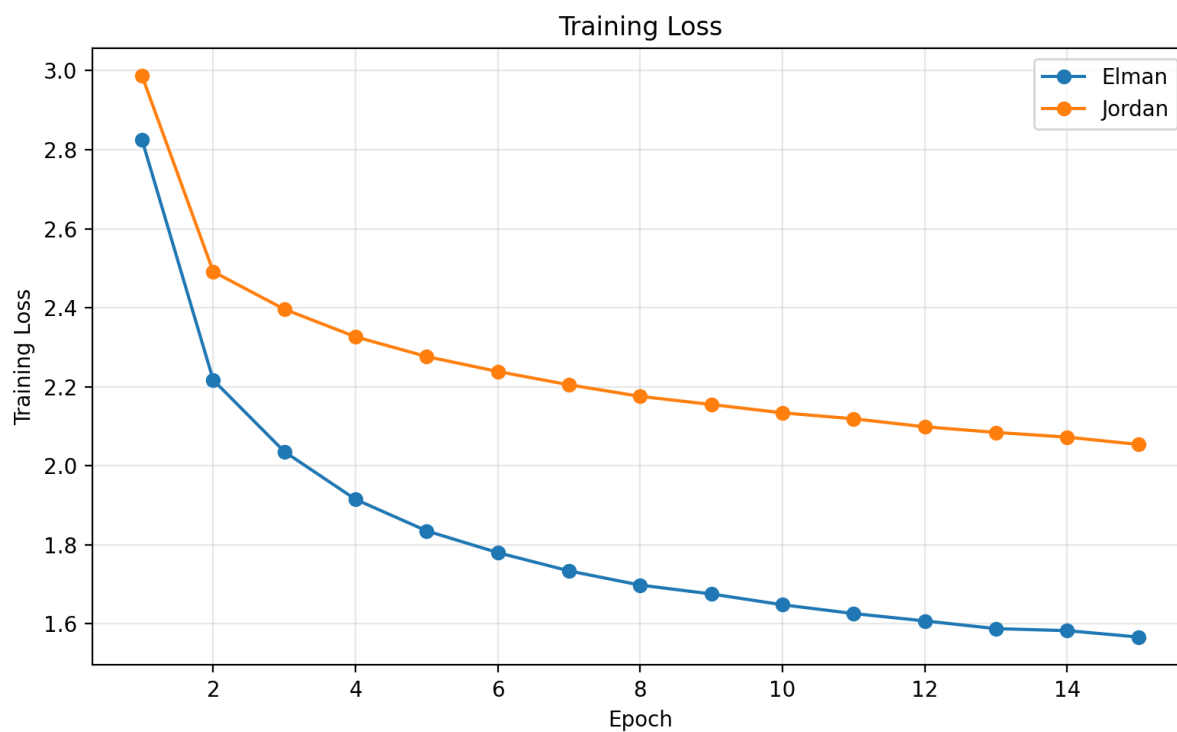
این Validation Accuracy. ضعیف‌تر بود Elman نیز در طول آموزش روند بهبودی داشت، اما عملکرد آن نسبت به Jordan مدل بود epoch 14 اول به 36.10٪ در پایان آموزش رسید و بهترین مقدار آن 36.23٪ در epoch مدل از 26.18٪ در

نیز از 14.04 به 9.16 رسید. این روند Validation PPL از 2.6422 به 2.2143 کاهش یافت و Jordan مدل Validation Loss بوده است Elman نیز یادگیری داشته، اما سرعت و کیفیت یادگیری آن کمتر از Jordan نشان می‌دهد

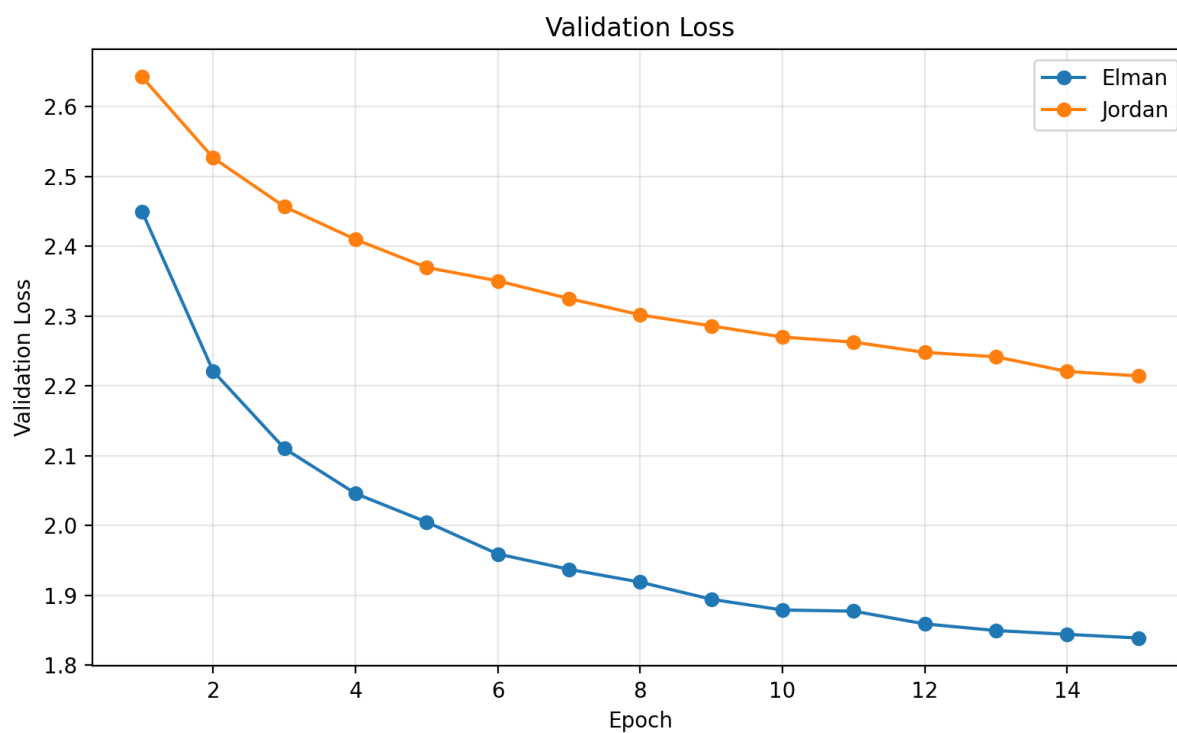
وارد مرحله بعد می‌شود. feedback خروجی قبلی مدل مستقیماً به عنوان Jordan یکی از دلایل احتمالی این موضوع این است که در اگر پیش‌بینی مرحله قبل خطا داشته باشد، این خطا می‌تواند روی گام‌های بعدی اثر بگذارد و روند یادگیری را دشوارتر کند

تحلیل نمودارها ۱۰.

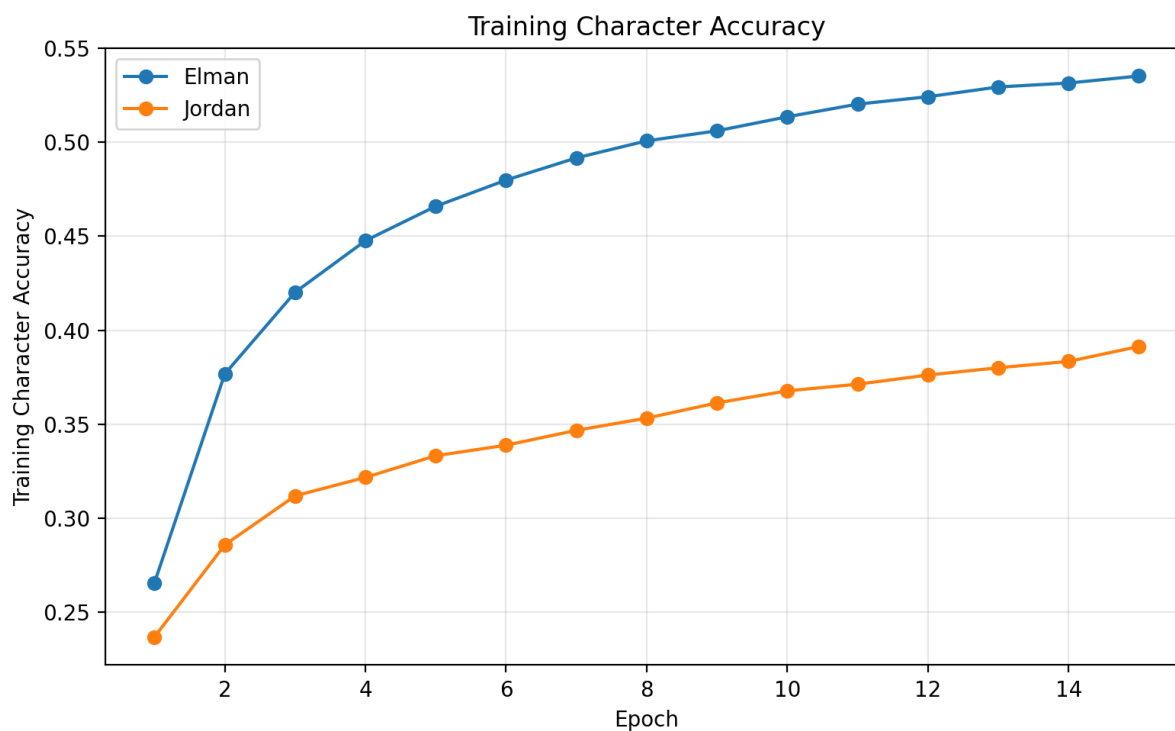
را برای هر دو مدل perplexity و accuracy، loss در این بخش شش نمودار اصلی خروجی کد بررسی می‌شوند. این نمودارها روند نشان می‌دهند validation و train در داده



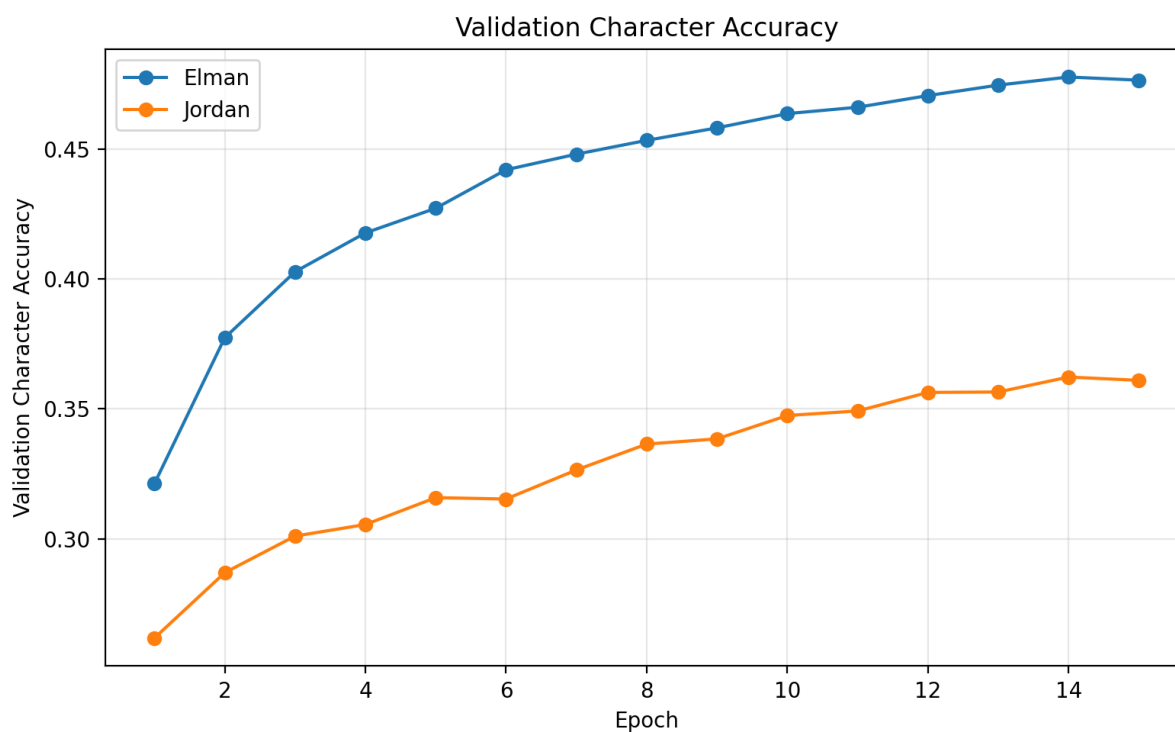
های آموزش epoch در طول Elman و Jordan برای مدل های Training Loss شکل 1. روند تغییر



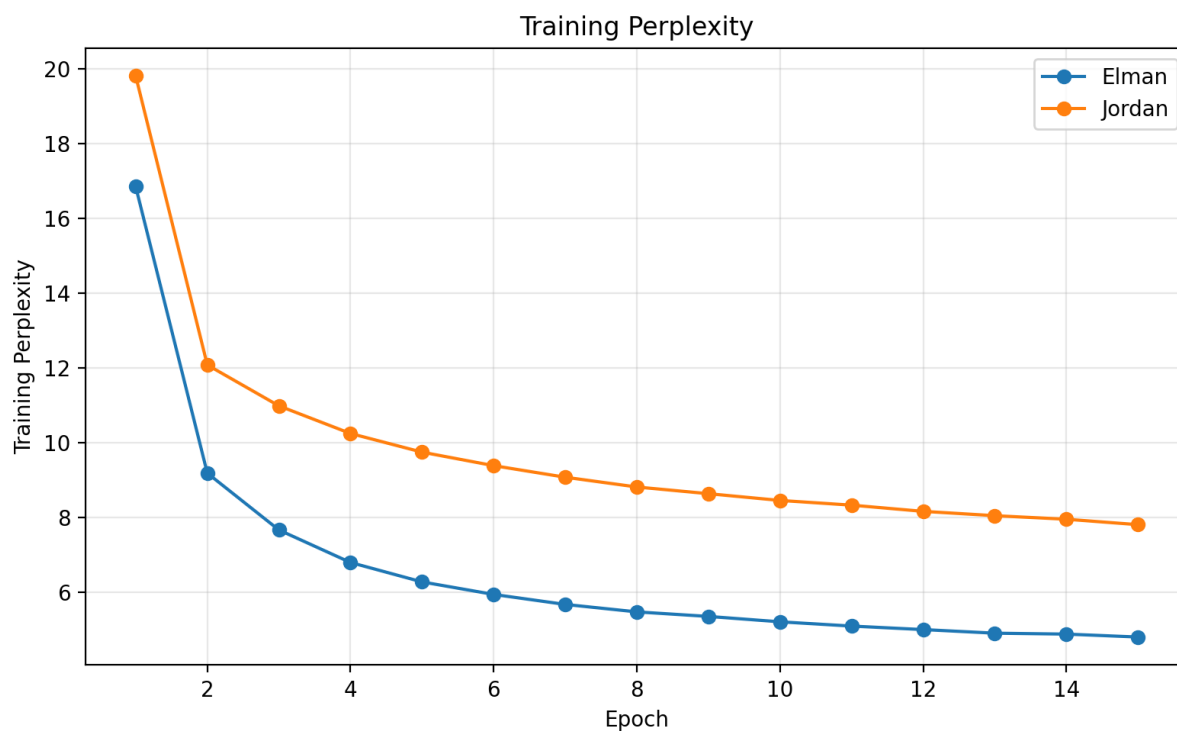
های آموزش epoch در طول Jordan و Elman برای مدل‌های Validation Loss شکل 2. روند تغییر



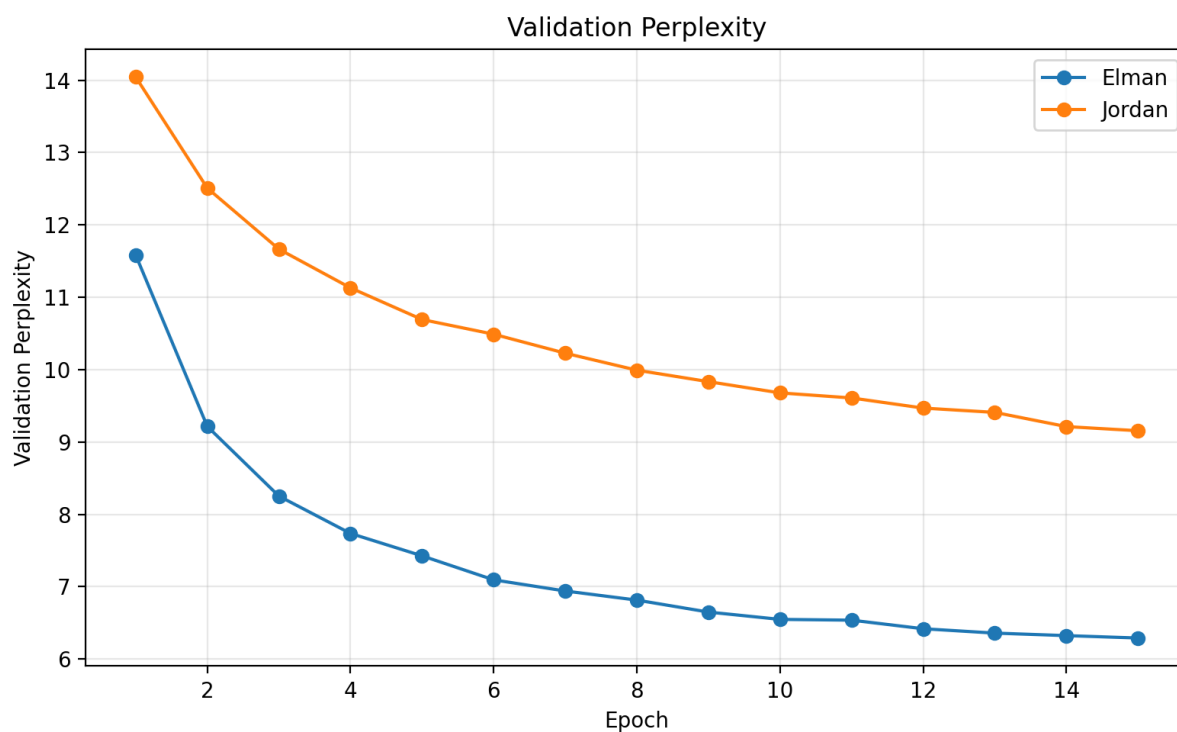
های مدل‌های Training Character Accuracy شکل 3. روند تغییر Jordan و Elman



Validation Character Accuracy مدل‌های Elman و Jordan شکل 4. روند تغییر



Training Perplexity مدل‌های Elman و Jordan شکل 5. روند تغییر



۱۰.۱ Loss تحلیل نمودارهای

در نمودار **Training Loss** مشاهده می‌شود که خطای آموزشی هر دو مدل در طول epoch ها کاهش یافته است. این موضوع نشان می‌دهد که هر دو مدل توانسته‌اند از داده‌های آموزشی یاد بگیرند. با این حال، کاهش loss در مدل **Elman** سریع‌تر بوده و مقدار نهایی آن نیز کمتر از مدل **Jordan** است.

مقدار **Train Loss** مدل **Elman** از **2.8245** به **1.5667** رسید، در حالی که **Train Loss** مدل **Jordan** از **2.9865** به **2.0541** کاهش پیدا کرد. بنابراین، مدل **Elman** روی داده‌های آموزشی عملکرد بهتری داشته است.

در نمودار **Validation Loss** نیز روند مشابهی مشاهده می‌شود. مقدار **Validation Loss** مدل **Elman** از **2.4496** به **1.8389** کاهش یافت، اما مقدار نهایی **Validation Loss** مدل **Jordan** برابر با **2.2143** بود.

بنابراین می‌توان نتیجه گرفت که مدل **Elman** علاوه بر یادگیری بهتر روی داده‌های آموزشی، روی داده‌های validation نیز عملکرد بهتری نسبت به مدل **Jordan** داشته است.

۱۰.۲ Accuracy تحلیل نمودارهای

با سرعت بیشتری افزایش پیدا کرده است. دقت **Elman** نشان می‌دهد دقت آموزشی **Training Character Accuracy** نمودار از **23.69٪** به **39.13٪** رسید **Jordan** از **26.57٪** به **53.51٪** رسید، در حالی که **Elman** آموزشی

Validation عمل کرده است **Jordan** ها بهتر از epoch در تمام **Elman** نیز **Validation Character Accuracy** در نمودار از **26.18٪** به **Jordan**، **32.13٪** به **47.65٪** رسید و بهترین مقدار آن **47.77٪** بود. در مقابل **Elman** مدل **Accuracy** رسید و بهترین مقدار آن **36.23٪** بود **36.10٪**

۱۰.۳ Perplexity تحلیل نمودارهای

یعنی مدل در پیش‌بینی کاراکتر بعدی کمتر دچار سردرگمی **PPL** هر دو مدل در طول آموزش کاهش یافته است. کاهش **Perplexity** است. **Jordan** ها کمتر از epoch در تمام **Elman** مدل **PPL** شده است. با این حال، مقدار

Jordan، آن از **11.58** به **6.29** کاهش یافت. در مقابل **PPL Validation** از **16.85** به **4.79** رسید و **Elman** مدل **PPL Training** را تأیید **Elman** نیز برتری مدل **PPL** رسید. بنابراین نمودارهای **validation** و از **14.04** به **9.16** در **train** از **19.82** به **7.80** در می‌کنند.

۱۱. مقایسه همگرایی و پایداری

با توجه به نتایج عددی و نمودارها، مدل **Elman** هم از نظر سرعت همگرایی و هم از نظر پایداری آموزش عملکرد بهتری نسبت به مدل **Jordan** داشته است.

مدل **Elman** خیلی زودتر به **loss** پایین‌تر و **accuracy** بالاتر رسید. در مقابل، مدل **Jordan** نیز روند یادگیری قابل مشاهده‌ای داشت، اما سرعت یادگیری آن کمتر بود و فاصله عملکرد آن با **Elman** تا پایان آموزش حفظ شد.

بنابراین می‌توان نتیجه گرفت که در این آزمایش، مدل **Elman** سریع‌تر و مؤثرتر از **Jordan** الگوهای متنی را یاد گرفته و در طول آموزش نیز عملکرد بهتری از خود نشان داده است.

مدل بهتر	معیار
Elman	Train Loss
Elman	Validation Loss
Elman	Train Accuracy
Elman	Validation Accuracy
Elman	Train Perplexity
Elman	Validation Perplexity
Elman	Convergence Speed
Elman	Stability

Temperature تولید متن و اثر ۱۲.

پس از آموزش، هر دو مدل با seed یکسان زیر شروع به تولید متن کردند:

ROMEO:

برای بررسی اثر میزان تصادفی بودن در فرآیند sampling ، سه مقدار مختلف temperature مورد استفاده قرار گرفت:

0.4 ، ۰/۸ ، ۱/۲

هدف از تغییر temperature این بود که رفتار دو مدل در شرایط مختلف تولید متن بررسی شود. مقدار temperature پایین باعث می‌شود مدل محافظه‌کارانه‌تر عمل کند و بیشتر کاراکترهایی را انتخاب کند که احتمال بالاتری دارند. در نتیجه، خروجی معمولاً پایدارتر و منظم‌تر است، اما ممکن است تکراری شود. در مقابل، temperature بالاتر باعث افزایش تصادفی بودن و تنوع خروجی می‌شود، اما احتمال تولید واژه‌های ناقص، نویز و جمله‌های کم‌انجام را نیز بیشتر می‌کند.

Temperature = 0.4 ۱۲.۱

در این حالت، هر دو مدل محافظه‌کارانه‌تر عمل می‌کنند و بیشتر به سمت انتخاب کاراکترهای پرتکرار و محتمل می‌روند. بنابراین خروجی‌ها نسبت به دماهای بالاتر پایدارتر هستند، اما احتمال تکرار در آن‌ها بیشتر است.

مدل: Elman

در خروجی Elman با temperature=0.4 ، ساختار کلی متن تا حدی شبیه متن نمایشی حفظ شده است. مدل توانسته پس از seed اولیه، نام شخصیتی مانند MENENIUS: تولید کند و سپس متن را به صورت دیالوگ ادامه دهد. اگرچه در خروجی واژه‌های ناقص و نامعتبر مانند prount، meed و manieve دیده می‌شود، اما ترتیب کلی متن، وجود نام شخصیت و حالت دیالوگی خروجی نشان می‌دهد که Elman بخشی از الگوهای ترتیبی دیتاست را یاد گرفته است. در این دما، خروجی Elman نسبتاً منظم‌تر، خواناتر و کم‌نویزتر است.

مدل: Jordan

در خروجی Jordan با temperature=0.4 ، مدل نیز تلاش کرده ساختاری شبیه متن نمایشی تولید کند، اما کیفیت خروجی ضعیف‌تر از Elman است. در نمونه تولیدشده، نام‌هایی مانند QUS و MENENGLARD ICINIUS دیده می‌شود که نسبت به نام‌های تولیدشده توسط Elman نامنظم‌تر و غیرطبیعی‌تر هستند. همچنین عبارت‌هایی مانند The of the the wer wer the have wis the and the the me نشان می‌دهد که Jordan در دمای پایین بیشتر به تکرار کلمات پرتکرار و ساختارهای ساده تمایل دارد. بنابراین اگرچه temperature پایین Jordan را تا حدی پایدار کرده، اما متن تولیدشده همچنان تکراری‌تر و کم‌معناتر از Elman است.

مقایسه در: temperature=0.4

در این دما، Elman عملکرد بهتری دارد؛ زیرا هم ساختار دیالوگ را بهتر حفظ کرده و هم خروجی آن نسبت به Jordan خواناتر است. Jordan بیشتر دچار تکرار و تولید واژه‌های نامعتبر شده است.

Temperature = 0.8۱۲.۲

در این حالت، میزان تصادفی بودن نسبت به $temperature=0.4$ بیشتر می‌شود. بنابراین مدل‌ها فقط محتمل‌ترین کاراکترها را انتخاب نمی‌کنند و امکان تولید خروجی متنوع‌تر فراهم می‌شود. این مقدار معمولاً تعادل مناسبی بین تنوع و انسجام ایجاد می‌کند.

مدل: Elman

در خروجی Elman با $temperature=0.8$ ، مدل توانسته نام شخصیتی مانند CORIOLANUS تولید کند و سپس متنی شبیه دیالوگ ادامه دهد. خروجی نسبت به $temperature=0.4$ متنوع‌تر است و ساختار آن کمتر تکراری به نظر می‌رسد. البته همچنان خطاهایی مانند واژه‌های ناقص یا نادرست، مثل trubel'd، دیده می‌شود و جمله‌ها از نظر زبانی کاملاً صحیح نیستند. با این حال، وجود نام شخصیت، حفظ نسبی قالب دیالوگ و تولید عبارت‌های نسبتاً خوانا نشان می‌دهد که Elman در این دما تعادل خوبی بین تنوع و انسجام ایجاد کرده است.

مدل: Jordan

در خروجی Jordan با $temperature=0.8$ ، تنوع نسبت به دمای ۰/۴ بیشتر می‌شود، اما این افزایش تنوع الزاماً به معنای بهتر شدن کیفیت متن نیست. Jordan در این دما همچنان تمایل به تولید واژه‌های ناقص، ترکیب‌های غیرطبیعی و جمله‌های کم‌انسجام دارد. با افزایش $temperature$ ، مدل Jordan بیشتر تحت تأثیر خطاهای قبلی خود قرار می‌گیرد؛ زیرا در این معماری خروجی قبلی به عنوان feedback وارد مرحله بعد می‌شود. بنابراین اگر در یک گام خروجی نامناسبی تولید شود، احتمال دارد این خطا در ادامه متن نیز اثر بگذارد.

مقایسه در: temperature=0.8

در این دما، Elman نسبت به Jordan خروجی خواناتر و منسجم‌تری تولید می‌کند. Elman بهتر توانسته ساختارهای شبیه دیالوگ و نام شخصیت‌ها را حفظ کند، در حالی که Jordan بیشتر دچار بی‌نظمی و تولید واژه‌های ناقص می‌شود. به همین دلیل، $temperature=0.8$ برای Elman حالت مناسبی ایجاد کرده است، چون هم تنوع خروجی افزایش یافته و هم انسجام تا حد قابل قبولی حفظ شده است.

Temperature = 1.2۱۲.۳

در این حالت، مدل با آزادی بیشتری کاراکتر بعدی را انتخاب می‌کند. بنابراین احتمال انتخاب کاراکترهای کم‌احتمال بیشتر می‌شود و خروجی تنوع بالاتری پیدا می‌کند. با این حال، این افزایش تنوع معمولاً باعث کاهش انسجام، افزایش نویز و تولید واژه‌های نامعتبر می‌شود.

مدل: Elman

در خروجی Elman با $temperature=1.2$ ، متن نسبت به دماهای پایین‌تر متنوع‌تر اما نامنظم‌تر می‌شود. در این حالت، مدل آزادی بیشتری در انتخاب کاراکترها دارد و همین موضوع باعث می‌شود خطاهای املائی، واژه‌های ناقص و ترکیب‌های غیرطبیعی بیشتر ظاهر شوند. با این وجود، Elman به دلیل استفاده از hidden state قبلی، همچنان نسبت به Jordan توانایی بیشتری در حفظ بخشی از ساختار ترتیبی متن دارد. بنابراین حتی در دمای بالا نیز خروجی Elman معمولاً قابل قبول‌تر از Jordan است، هرچند کیفیت آن نسبت به $temperature=0.8$ کاهش پیدا می‌کند.

مدل: Jordan

در خروجی Jordan با $temperature=1.2$ ، بی‌نظمی و نویز بیشتر از حالت‌های قبلی دیده می‌شود. چون $temperature$ بالا باعث انتخاب‌های تصادفی‌تر می‌شود، احتمال تولید کاراکترها و واژه‌های نامناسب افزایش پیدا می‌کند. از طرف دیگر، در مدل Jordan خروجی قبلی مدل به عنوان feedback استفاده می‌شود؛ بنابراین اگر یک خروجی اشتباه یا نامنظم تولید شود، همان خطا می‌تواند روی گام‌های بعدی اثر بگذارد. به همین دلیل، Jordan در دمای بالا بیشتر از Elman دچار افت انسجام می‌شود و متن تولیدشده آن معمولاً خوانایی کمتری دارد.

مقایسه در: temperature=1.2

در این دما، هر دو مدل نسبت به دماهای پایین‌تر خروجی نویزتر و کم‌انسجام‌تری تولید می‌کنند. با این حال، افت کیفیت در Jordan شدیدتر است Elman. هنوز تا حدی می‌تواند ساختار کلی متن را حفظ کند، اما Jordan بیشتر دچار واژه‌های نامعتبر، ترکیب‌های تصادفی و جمله‌های نامفهوم می‌شود. بنابراین در دمای بالا نیز Elman عملکرد قابل‌قبول‌تری نسبت به Jordan دارد.

جمع‌بندی اثر Temperature

بررسی temperature نشان داد که مقدار ۰/۴ خروجی پایدارتر اما تکراری‌تر تولید می‌کند، مقدار ۰/۸ تعادل بهتری بین تنوع و انسجام دارد، و مقدار ۱/۲ باعث افزایش تنوع اما کاهش خوانایی و انسجام می‌شود. در هر سه حالت، مدل Elman نسبت به Jordan خروجی منسجم‌تر و طبیعی‌تری تولید کرد. این نتیجه با معیارهای عددی نیز همخوان است؛ زیرا Elman در validation accuracy، validation loss و validation perplexity عملکرد بهتری نسبت به Jordan داشت.

نمونه‌هایی از متن تولیدشده ۱۲.۴

در ادامه، چند نمونه کوتاه از متن‌های تولیدشده توسط مدل‌های Elman و Jordan آورده شده است. برای اینکه مقایسه بین دو مدل منصفانه باشد، تولید متن در هر دو مدل با seed یکسان زیر آغاز شد:

ROMEO:

همچنین برای بررسی تأثیر میزان تصادفی بودن در تولید متن، از سه مقدار مختلف temperature یعنی ۰/۴، ۰/۸ و ۱/۲ استفاده شد.

Elman, temperature=0.4

ROMEO:

MENENIUS:

Shall the stand their good for an and the prount the place, what I state to meed revort,

And show the manieve the mannot hear of the said to see him...

Elman, temperature=0.8

ROMEO:

CORIOLANUS:

I have not should be the dealt. I at trubel'd of your wan, when have best fin he were up...

Elman, temperature=1.2

ROMEO:

LARTIUS:

Why, then, the crown'd of breath? I speak no more,

But yet the people, thus in wildness, cry and turn away...

Jordan, temperature=0.4

ROMEO:

QUS:

MENENGLARD ICINIUS:

CORINGHARD:

The of the the wer wer the have wis the and the the me...

Jordan, temperature=0.8

ROMEO:

MENENIUS:

The lord the shall in the come and the men,

I have the word and there the state to him...

Jordan, temperature=1.2

ROMEO:

CAR:

Whar the murd of thee! unot sear, brath'd,

Qenfor no the hame, and his worle to brin...

نمونه‌ها نشان می‌دهند که خروجی Elman در دماهای پایین و متوسط منسجم‌تر است و ساختار دیالوگ و نام شخصیت‌ها را بهتر حفظ می‌کند. در $temperature=0.4$ خروجی پایدارتر اما تا حدی تکراری است. در $temperature=0.8$ تعادل بهتری بین تنوع و خوانایی دیده می‌شود. در $temperature=1.2$ تنوع متن بیشتر می‌شود، اما خطاهای املائی، واژه‌های ناقص و بی‌نظمی نیز افزایش پیدا می‌کند. در مدل Jordan این مشکل شدیدتر است و به‌خصوص در دماهای بالاتر، متن تولیدشده نویزی‌تر و کم‌انسجام‌تر از Elman است.

جمع‌بندی تحلیلی ۱۳.

با توجه به نتایج به‌دست آمده، می‌توان گفت که در این آزمایش، معماری Elman برای مسئله پیش‌بینی کاراکتر بعدی روی دیتاست Tiny Shakespeare مناسب‌تر بوده است. این مدل در معیارهای اصلی ارزیابی، عملکرد بهتری نسبت به مدل Jordan نشان داد.

مدل Elman توانست به $accuracy$ بالاتر، $loss$ کمتر و $perplexity$ پایین‌تر برسد. همچنین روند یادگیری آن سریع‌تر بود و در تولید متن نیز خروجی‌های منسجم‌تر و طبیعی‌تری ایجاد کرد.

در مقابل، مدل Jordan اگرچه توانست الگوهای متنی را تا حدی یاد بگیرد، اما هم از نظر معیارهای عددی و هم از نظر کیفیت متن تولیدشده، عملکرد ضعیف‌تری نسبت به Elman داشت.

این نتایج نشان می‌دهد که برای این دیتاست و این تنظیمات آزمایش، استفاده از hidden-state feedback در مدل Elman مؤثرتر از استفاده از output-distribution feedback در مدل Jordan بوده است. بنابر این می‌توان نتیجه گرفت که نوع feedback در شبکه‌های بازگشتی نقش مهمی در کیفیت یادگیری و عملکرد نهایی مدل دارد.

نتیجه‌گیری نهایی ۱۴.

در این پروژه، دو مدل بازگشتی Elman و Jordan برای پیش‌بینی کاراکتر بعدی روی دیتاست Tiny Shakespeare به صورت character-level پیاده‌سازی و مقایسه شدند. برای کاهش زمان آموزش، از ۳۰٪ دیتاست استفاده شد و هر دو مدل با تنظیمات یکسان آموزش داده شدند تا مقایسه بین آن‌ها منصفانه باشد.

نتایج به دست آمده نشان داد که مدل Elman عملکرد بهتری نسبت به مدل Jordan دارد. در پایان آموزش، مدل Elman به Validation Accuracy برابر با ۴۷.۶۵٪ و Validation Perplexity برابر با ۶.۲۹ رسید. در مقابل، مدل Jordan به Validation Accuracy برابر با ۳۶.۱۰٪ و Validation Perplexity برابر با ۹.۱۶ دست پیدا کرد. همچنین بهترین مقدار Validation Accuracy برای مدل Elman برابر با ۴۷.۷۷٪ و برای مدل Jordan برابر با ۳۶.۲۳٪ بود.

این نتایج نشان می‌دهد که استفاده از hidden state قبلی به عنوان حافظه در مدل Elman، برای مسئله پیش‌بینی کاراکتر بعدی مؤثرتر از استفاده از خروجی قبلی در مدل Jordan بوده است. به همین دلیل، مدل Elman توانست الگوهای متنی را بهتر یاد بگیرد و روی داده‌های validation نیز عملکرد بهتری نشان دهد.

علاوه بر معیارهای عددی، بررسی نمونه‌های تولید متن نیز نشان داد که مدل Elman در بیشتر دماها خروجی منسجم‌تر و طبیعی‌تری تولید می‌کند. در مقابل، خروجی‌های مدل Jordan بیشتر دچار تکرار، بی‌نظمی و ساختارهای نامفهوم می‌شوند. در مجموع، این تمرین نشان داد که نوع feedback در شبکه‌های بازگشتی تأثیر قابل توجهی بر کیفیت یادگیری، عملکرد validation و کیفیت متن تولید شده دارد.

پیوست: خروجی اجرای کد در ترمینال

در محیط ترمینال را نشان می‌دهند. این خروجی‌ها مبنای p2_completed.py تصاویر زیر بخشی از خروجی واقعی اجرای کد جدول‌ها و تحلیل‌های عددی گزارش هستند.


```
PS C:\Users\Administrator\Downloads> & 'C:\Users\Administrator\AppData\Local\Python\Python39\python.exe' 'C:\Users\Administrator\AppData\Local\Python\Python39\python\python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '60906' '--' 'C:\Users\Administrator\Downloads\Tiny Shakespeare Memory Duel - Completed p2.py'
```

```
=====
Device: cpu
Dataset fraction: 0.30 = 30%
Dataset path: C:\Users\Administrator\Downloads\tiny-shakespeare(1).txt
Characters used: 334,618
Vocabulary size: 63
Train characters: 301,156 | Validation characters: 33,462
```

شکل 7. تنظیمات اولیه اجرا و اطلاعات دیتاست در ترمینال

```
=====
Training Elman Network
=====
Epoch 01/15 | Train Loss: 2.8245 | Train Acc: 0.2657 | Train PPL: 16.85 | Val Loss: 2.4496 | Val Acc: 0.3213 | Val PPL: 11.58 | Time: 6.9s | best saved
Epoch 02/15 | Train Loss: 2.2171 | Train Acc: 0.3767 | Train PPL: 9.18 | Val Loss: 2.2208 | Val Acc: 0.3774 | Val PPL: 9.21 | Time: 6.7s | best saved
Epoch 03/15 | Train Loss: 2.0358 | Train Acc: 0.4202 | Train PPL: 7.66 | Val Loss: 2.1104 | Val Acc: 0.4028 | Val PPL: 8.25 | Time: 8.8s | best saved
Epoch 04/15 | Train Loss: 1.9150 | Train Acc: 0.4476 | Train PPL: 6.79 | Val Loss: 2.0458 | Val Acc: 0.4177 | Val PPL: 7.74 | Time: 8.7s | best saved
Epoch 05/15 | Train Loss: 1.8352 | Train Acc: 0.4659 | Train PPL: 6.27 | Val Loss: 2.0049 | Val Acc: 0.4272 | Val PPL: 7.43 | Time: 8.7s | best saved
Epoch 06/15 | Train Loss: 1.7802 | Train Acc: 0.4798 | Train PPL: 5.93 | Val Loss: 1.9592 | Val Acc: 0.4420 | Val PPL: 7.09 | Time: 8.6s | best saved
Epoch 07/15 | Train Loss: 1.7340 | Train Acc: 0.4916 | Train PPL: 5.66 | Val Loss: 1.9373 | Val Acc: 0.4480 | Val PPL: 6.94 | Time: 8.7s | best saved
Epoch 08/15 | Train Loss: 1.6981 | Train Acc: 0.5007 | Train PPL: 5.46 | Val Loss: 1.9190 | Val Acc: 0.4533 | Val PPL: 6.81 | Time: 8.7s | best saved
Epoch 09/15 | Train Loss: 1.6757 | Train Acc: 0.5060 | Train PPL: 5.34 | Val Loss: 1.8944 | Val Acc: 0.4582 | Val PPL: 6.65 | Time: 9.2s | best saved
Epoch 10/15 | Train Loss: 1.6483 | Train Acc: 0.5135 | Train PPL: 5.20 | Val Loss: 1.8791 | Val Acc: 0.4637 | Val PPL: 6.55 | Time: 9.0s | best saved
Epoch 11/15 | Train Loss: 1.6260 | Train Acc: 0.5202 | Train PPL: 5.08 | Val Loss: 1.8774 | Val Acc: 0.4661 | Val PPL: 6.54 | Time: 8.8s | best saved
Epoch 12/15 | Train Loss: 1.6074 | Train Acc: 0.5241 | Train PPL: 4.99 | Val Loss: 1.8591 | Val Acc: 0.4706 | Val PPL: 6.42 | Time: 8.7s | best saved
Epoch 13/15 | Train Loss: 1.5879 | Train Acc: 0.5293 | Train PPL: 4.89 | Val Loss: 1.8497 | Val Acc: 0.4746 | Val PPL: 6.36 | Time: 8.7s | best saved
Epoch 14/15 | Train Loss: 1.5828 | Train Acc: 0.5314 | Train PPL: 4.87 | Val Loss: 1.8442 | Val Acc: 0.4777 | Val PPL: 6.32 | Time: 8.9s | best saved
Epoch 15/15 | Train Loss: 1.5667 | Train Acc: 0.5351 | Train PPL: 4.79 | Val Loss: 1.8389 | Val Acc: 0.4765 | Val PPL: 6.29 | Time: 9.1s | best saved
```

در ترمینال Elman شکل 8. خروجی آموزش مدل

```
=====
Training Jordan Network
=====
Epoch 01/15 | Train Loss: 2.9865 | Train Acc: 0.2369 | Train PPL: 19.82 | Val Loss: 2.6422 | Val Acc: 0.2618 | Val PPL: 14.04 | Time: 9.3s | best saved
Epoch 02/15 | Train Loss: 2.4914 | Train Acc: 0.2859 | Train PPL: 12.08 | Val Loss: 2.5265 | Val Acc: 0.2870 | Val PPL: 12.51 | Time: 11.4s | best saved
Epoch 03/15 | Train Loss: 2.3963 | Train Acc: 0.3119 | Train PPL: 10.98 | Val Loss: 2.4566 | Val Acc: 0.3010 | Val PPL: 11.67 | Time: 10.9s | best saved
Epoch 04/15 | Train Loss: 2.3265 | Train Acc: 0.3218 | Train PPL: 10.24 | Val Loss: 2.4097 | Val Acc: 0.3055 | Val PPL: 11.13 | Time: 11.6s | best saved
Epoch 05/15 | Train Loss: 2.2765 | Train Acc: 0.3333 | Train PPL: 9.74 | Val Loss: 2.3695 | Val Acc: 0.3158 | Val PPL: 10.69 | Time: 9.7s | best saved
Epoch 06/15 | Train Loss: 2.2386 | Train Acc: 0.3389 | Train PPL: 9.38 | Val Loss: 2.3504 | Val Acc: 0.3153 | Val PPL: 10.49 | Time: 9.1s | best saved
Epoch 07/15 | Train Loss: 2.2050 | Train Acc: 0.3468 | Train PPL: 9.07 | Val Loss: 2.3250 | Val Acc: 0.3265 | Val PPL: 10.23 | Time: 9.2s | best saved
Epoch 08/15 | Train Loss: 2.1758 | Train Acc: 0.3533 | Train PPL: 8.81 | Val Loss: 2.3018 | Val Acc: 0.3365 | Val PPL: 9.99 | Time: 9.3s | best saved
Epoch 09/15 | Train Loss: 2.1552 | Train Acc: 0.3614 | Train PPL: 8.63 | Val Loss: 2.2858 | Val Acc: 0.3385 | Val PPL: 9.83 | Time: 11.0s | best saved
Epoch 10/15 | Train Loss: 2.1340 | Train Acc: 0.3678 | Train PPL: 8.45 | Val Loss: 2.2699 | Val Acc: 0.3474 | Val PPL: 9.68 | Time: 9.8s | best saved
Epoch 11/15 | Train Loss: 2.1190 | Train Acc: 0.3713 | Train PPL: 8.32 | Val Loss: 2.2626 | Val Acc: 0.3492 | Val PPL: 9.61 | Time: 9.9s | best saved
Epoch 12/15 | Train Loss: 2.0988 | Train Acc: 0.3762 | Train PPL: 8.16 | Val Loss: 2.2479 | Val Acc: 0.3563 | Val PPL: 9.47 | Time: 9.8s | best saved
Epoch 13/15 | Train Loss: 2.0845 | Train Acc: 0.3800 | Train PPL: 8.04 | Val Loss: 2.2416 | Val Acc: 0.3565 | Val PPL: 9.41 | Time: 9.8s | best saved
Epoch 14/15 | Train Loss: 2.0726 | Train Acc: 0.3834 | Train PPL: 7.95 | Val Loss: 2.2206 | Val Acc: 0.3623 | Val PPL: 9.21 | Time: 9.7s | best saved
Epoch 15/15 | Train Loss: 2.0541 | Train Acc: 0.3913 | Train PPL: 7.80 | Val Loss: 2.2143 | Val Acc: 0.3610 | Val PPL: 9.16 | Time: 9.8s | best saved
```

در ترمینال Jordan شکل 9. خروجی آموزش مدل